

# If statements

INTERMEDIATE JAVA



**Jim White**  
Java Developer

# Control flow

If the condition is true → do something

If the condition is false → do nothing

# How if statements work

If score is 90+ ➡ print "Great job!"

If score is less than 90 ➡ don't print it

# if statement anatomy

```
if (condition) {  
    // Code to run  
}
```

If the condition is true, the code inside curly brackets runs

# Conditions

- Use comparison operators like `==` , `!=` , `>` , `<` , `>=` , `<=` , ...

```
// Print "Great job!" if score is >= 90
if (score >= 90) {
    System.out.println("Great job!");
}
```

# if and if and if ...

```
int score = 94;

if (score >= 90) {
    System.out.println("Excellent!");
}

if (score >= 70) {
    System.out.println("Good job!");
}

if (score < 70) {
    System.out.println("Try again!");
}
```

Excellent!  
Good job!

- Both messages get displayed because the two conditions are satisfied

# Recap

We can have different conditions, if they evaluate to `true` / `false`

```
if (score != 0){  
    System.out.println("Well, you got some points!");  
}  
  
if (message.equals("F")){  
    System.out.println("Try again!");  
}  
  
if (testResult == 100) {  
    System.out.println("Wow, you got it all!");  
}
```

**Let's practice!**  
INTERMEDIATE JAVA

# Else, else if

INTERMEDIATE JAVA



**Jim White**  
Java Developer

# Multiple ifs

```
if (score >= 90) {  
    System.out.println("Excellent!");  
}  
  
if (score >= 70) {  
    System.out.println("Good effort.");  
}  
  
if (score < 70) {  
    System.out.println("Keep trying!");  
}
```

If `score >= 70`, both `Excellent!` and `Good effort.` are printed

➔ Using multiple if statements can get messy

# if - else if - else

```
int score = 80;

if (score >= 90) {
    System.out.println("Excellent!");
} else if (score >= 70) {
    // Only this code is executed
    System.out.println("Good effort.");
} else {
    System.out.println("Keep trying!");
}
```

Good effort.

```
int score = 60;

if (score >= 90) {
    System.out.println("Excellent!");
} else if (score >= 70) {
    System.out.println("Good effort.");
} else { // Only this code is executed
    System.out.println("Keep trying!");
}
```

Keep trying!

# Only one block is run

```
int score = 95;

if (score >= 90) { // Only this code is executed
    System.out.println("Excellent!");
} else if (score >= 70) {
    System.out.println("Good effort.");
} else {
    System.out.println("Keep trying!");
}
```

Excellent!

# As many as we need - just be careful about the order

```
// As many as we want
if (score >= 95) {
    System.out.println("Excellent job, well c
} else if (score >= 90) {
    System.out.println("Excellent!");
} else if (score >= 80) {
    System.out.println("Great job.");
} else if (score >= 70) {
    System.out.println("Good effort.");
} else {
    System.out.println("Keep trying!");
}
```

```
// Be careful with the order
int score = 95;

if (score > 70) { // This code runs
    System.out.println("Good effort.");
} else if (score >= 90) {
    // This is also valid
    System.out.println("Excellent!");
} else {
    System.out.println("Keep trying!");
}
```

Good effort.

# Recap

```
int age = 33;

if (age < 20) {
    System.out.println("You are a teenager");
} else if (age < 30) {
    System.out.println("You are in your twenties");
} else if (age < 40) { // Only this code block will be executed
    System.out.println("You are in your thirties");
} else {
    System.out.println("You are older than 39");
}
```

You are in your thirties

# Let's practice!

INTERMEDIATE JAVA

# Logical operators

INTERMEDIATE JAVA



**Jim White**  
Java Developer

# ARE, OR, and NOT

- Logical operators allow combining of multiple conditions in one statement
- Three main ones:
  - AND
  - OR
  - NOT

# AND

- Checks if both conditions are true

```
boolean isLoggedIn = true;
boolean isAdmin = true;

// Both are true
if (isLoggedIn && isAdmin) {
    System.out.println("Welcome admin!");
} else {
    System.out.println("You shall not pass.");
}
```

Welcome admin!

- If at least one is false, `&&` returns false

```
boolean isLoggedIn = false;
boolean isAdmin = true;

// Only one is true, so else is executed
if (isLoggedIn && isAdmin) {
    System.out.println("Welcome admin!");
} else {
    System.out.println("You shall not pass.");
}
```

You shall not pass.

# OR

- Checks if either condition is `true`

```
boolean isAdmin = true;
boolean isModerator = false;

// Only one condition is true
if (isAdmin || isModerator) {
    System.out.println("Welcome!");
} else {
    System.out.println("You shall not pass.");
}
```

Welcome!

- `true` if either or both conditions are `true`

```
boolean isAdmin = true;
boolean isModerator = true;

// Both conditions are true
if (isAdmin || isModerator) {
    System.out.println("Welcome!");
} else {
    System.out.println("You shall not pass.");
}
```

Welcome!

# NOT

- Flips `true` to `false` and `false` to `true`

```
boolean isLoggedIn = false;

// isLoggedIn is false, so !isLoggedIn is true
if (!isLoggedIn) {
    System.out.println("You need to log in!");
} else {
    System.out.println("You are logged in, continue!");
}
```

You need to log in!

# Logical operators and if-else if-else

```
int score = 90;
boolean isAttending = false;

if (score > 80 && isAttending) {
    System.out.println("Excellent work!");
} else if (score > 80 && !isAttending) {
    System.out.println("Good work, but attendance is important.");
} else if (score <= 80 && isAttending) {
    System.out.println("You need to work harder.");
} else {
    System.out.println("You need to work harder and attend classes.");
}
```

Good work, but attendance is important.

# Recap

The following table summarizes the values for `&&` , `||` , and `!` operators:

| a     | b     | a && b | a    b | !a    |
|-------|-------|--------|--------|-------|
| true  | true  | true   | true   | false |
| true  | false | false  | true   | false |
| false | true  | false  | true   | true  |
| false | false | false  | false  | true  |

- Logical operators can be used with booleans or with expressions that evaluate to booleans

```
boolean a = true && (!false || (true && false)); // a is true
boolean b = (7 <= 3) || (3 >= 5); // b is false
```

**Let's practice!**  
INTERMEDIATE JAVA

# Switch statements

INTERMEDIATE JAVA



**Jim White**  
Java Developer

# What is a switch?

- Switch serves a purpose similar to `if-else` statements
  - Does some actions based on some condition
  - Used in place of multiple if-then-else conditionals
- Considered more concise than multiple if-then-else

```
if (bonus == 10000) {  
    System.out.println("Let's buy a car!");  
}  
else if (bonus == 5000) {  
    System.out.println("Let's take a trip!");  
}  
else if (bonus == 1000) {  
    System.out.println("Let's save!");  
}
```

# Switch syntax

```
switch (expression) {  
    case value1: // if (expression == value1)  
        // statements  
    case value2: // if (expression == value2)  
        // statements  
    // use as many case statements as needed  
}
```

# Switch example

```
char direction = 'N';

switch (direction) {
    case 'E':
        System.out.println("We are headed east.");
    case 'S':
        System.out.println("We are headed south.");
    case 'W':
        System.out.println("We are headed west.");
    case 'N':
        System.out.println("We are headed north."); // This would be printed
}
```

# Break

- Stops Java continuing through the rest of cases
  - Without break, Java execute statements in the next case until it hits a break

```
char direction = 'W';

switch (direction) {
    case 'E':
        System.out.println("We are headed east.");
    case 'S':
        System.out.println("We are headed south.");
    case 'W':
        System.out.println("We are headed west."); // This would be printed
    case 'N':
        System.out.println("We are headed north."); // This would also be printed
}
```

**break** allows combining multiple cases together:

```
char direction = 'N';

switch (direction) {
    case 'E':
    case 'W':
        System.out.println("We are headed east or west.");
        break;
    case 'S':
    case 'N':
        System.out.println("We are headed south or north.");
        break;
}
```

We are headed south or north.

# Default

- An optional `default` case can be added to the end of a switch
  - The default case has no value to compare to the expression
  - If no case value matches the expression, the default statements get executed
  - `default` must be the last case in a switch
- `default` is a way to have an "all other values" case

```
char direction = 'z';

switch (direction) {
    case 'E':
    case 'W':
        System.out.println("To east or west.");
        break;
    case 'S':
    case 'N':
        System.out.println("To south or north.");
        break;
    default:
        System.out.println("We are lost.");
}
```

We are lost.

# Conditionals versus switch

- Conditionals could be used to complete the same task
- Work with every data type
- Switch can be easier to read
- Works only with some data types like `int`, `byte`, `short`, `char`, or `String`

```
char character = 'A';
if (character == 'A') {
    System.out.println("It's A");
} else if (direction == 'B') {
    System.out.println("It's B");
} else {
    System.out.println("Unknown");
}
```

```
char character = 'A';
switch (character) {
    case 'A':
        System.out.println("It's A");
        break;
    case 'B':
        System.out.println("It's B");
        break;
    default:
        System.out.println("Unknown");
}
```

# Let's practice!

INTERMEDIATE JAVA